Beslutningslog

Her er de vigtigste tekniske valg i løsningen, hvilke alternativer jeg overvejede, og hvorfor jeg valgte, som jeg gjorde. Fortolkninger af opgaven, hvor teksten ikke var entydig, står i et afsnit for sig.

Udgangspunkt

Templaten: behold strukturen, skriv domænet forfra

Valg: Jeg beholdt templatens lagdeling (Domain, Application, Infrastructure, WebAPI og Tests) og React-opsætningen, men slettede og genskrev entiteter, services og controllere.

Hvorfor: Templaten indeholdt kode fra et andet projekt, som ikke passede til opgaven.

Alternativ: Starte forfra uden templaten. Fravalgt, fordi solution, projektreferencer, testprojekt og frontend-opsætning allerede var på plads, og fordi templaten viser teamets egen stack.

.NET 8 frem for .NET 10

Valg: Projektet kører stadig på .NET 8, som templaten er sat op til.

Hvorfor: Det er den version, teamet har sat op, så jeg undgik risikoen for opsætningsproblemer under tidspres.

Konsekvens: .NET 8 har end of support den 10. november 2026. En opgradering til .NET 10 (LTS) er det første, jeg ville gøre, hvis løsningen skulle videre. Det kræver at ændre `TargetFramework` til `net10.0` og opdatere pakkerne.

Ingen login

Valg: Løsningen har ingen autentificering. Adgangsstyring betyder her at administrere brugere og deres roller.

Hvorfor: Opgaven beder om at registrere roller, ikke om login, og siger, at felterne på en bruger er navn, e-mail, Admin og Personale. Login ville kræve passwords, tokens, en login-side og beskyttede endpoints, som ikke er beskrevet.

I produktion: Jeg ville ikke bygge eget login, men koble på organisationens eksisterende login via OpenID Connect (fx Microsoft Entra ID) og bruge Admin-rollen til at styre adgang i API'et.

Backend

Database: SQLite

Valg: SQLite som fil (`vagtplan.db`), der oprettes automatisk ved første opstart.

Hvorfor: Løsningen kan køres med det samme på enhver maskine uden at installere eller konfigurere en databaseserver.

Alternativ: SQL Server, som teamet bruger. Fordi al dataadgang går gennem EF Core, er et skift én linje (`UseSqlite` til `UseSqlServer`), en ny connection string og en ny migration.

Begrænsning: SQLite låser hele databasen ved skrivning og egner sig ikke til mange samtidige brugere. I produktion ville jeg bruge SQL Server.

Ingen repository-lag

Valg: Services bruger et `IAppDbContext`-interface, som Application definerer, og Infrastructure implementerer.

Hvorfor: De fleste af opgavens regler afhænger af databasens tilstand: overlap, unik e-mail og "må ikke slettes, hvis der er ansættelser". Tester man services med mocks af et repository, bestemmer testen selv, hvad repositoriet svarer, og selve forespørgslerne bliver ikke testet. Microsofts vejledning til EF Core anbefaler at teste mod en rigtig database og beskriver repository pattern som vejen, hvis man har brug for test doubles.

Alternativ: Et repository pr. Entitet.Det giver renere lag og services, der kan unit-testes med mocks, men kræver flere filer og separate integrationstests af forespørgslerne.

Konsekvens: Application-laget kender EF Core. Det er en bevidst afvejning, som også ses i udbredte Clean Architecture-skabeloner til .NET.

Tests: unit og integration adskilt

Valg: - `Tests/Unit`: rene regler uden database (`DateRange`, `HexColor`, `EmailAddress`) - `Tests/Integration`: services og validatorer mod en SQLite-database i hukommelsen, en ny og tom database pr. test

Hvorfor: Jeg har tidligere forsøgt mig med EF Core's InMemory-provider, som ikke håndhæver fremmednøgler, unikke indeks eller andre constraints. SQLite in-memory er en rigtig relationel database, der håndhæver constraints, og den skarpe opdeling gør det tydeligt, hvad hver test tester.

Begrænsning: Skiftede produktionen til SQL Server, ville testene køre mod en anden database end produktionen. Næste skridt ville så være integrationstests mod SQL Server i teamets pipeline.

Datoer: `DateOnly` og inklusive datoer

Valg: Perioder og ansættelser bruger `DateOnly`, og både start- og slutdato tæller med.

Hvorfor: Det er hele kalenderdage. `DateTime` ville tage klokkeslæt og tidszoner med og kunne flytte en dato. Opgavens eget eksempel (December 2026: 01-12 til 31-12 og Januar 2027: 01-01 til 31-01) viser, at slutdatoen tæller med.

Overlap-reglen ét sted

Valg: Reglen `startA <= endB && startB <= endA` ligger i `Core/Rules/DateRange.cs` og er unit-testet med grænsetilfælde, herunder opgavens eksempel og to intervaller, der deler én dag.

Hvorfor: Reglen bruges tre steder (perioder, ansættelser og Personale-siden). Ligger den ét sted, skal den kun rettes ét sted.

Afvejning: EF Core kan ikke oversætte en C#-metode til SQL. Derfor: - **Perioder og ansættelser** henter de relevante rækker og sammenligner med `DateRange.Overlaps` i C#. Det er acceptabelt, fordi der er få perioder, og fordi ansættelser først filtreres på personen i databasen. - **Personale-siden** skal søge i alle ansættelser, et tal der vokser over tid. Her står betingelsen direkte i forespørgslen, så databasen filtrerer. Reglen står dermed to steder, og derfor tester `StaffServiceTests` præcis de samme grænsetilfælde som `DateRangeTests`.

Forretningsregler: validatorer og database

Valg: Hver entitet har en validator i Application, der samler alle fejl og kaster én `ValidationException`. Fælles regler (navn og datoer) ligger i `NameValidation` og `DateValidation`. Databasen har constraints som sikkerhedsnet.

Regel	Validator	Database
Påkrævede felter	Ja	NOT NULL
Unik e-mail	Ja	Unikt indeks
Timer 0–37, start før slut	Ja	CHECK-constraints
Må ikke slettes med ansættelser	Ja (i servicen)	Fremmednøgle med `Restrict`
Ingen overlap	Ja	Kan ikke udtrykkes som en almindelig constraint

Hvorfor: Validatoren giver brugeren en forståelig dansk besked ved det rigtige felt. Databasen fanger det, hvis noget alligevel slipper igennem. Validatorerne blev flyttet ud af servicerne, så hver klasse har ét ansvar.

Fejlhåndtering: egne exceptions og én central handler

Valg: Application kaster `ValidationException`, `NotFoundException` og `ConflictException`. `GlobalExceptionHandler` i WebAPI oversætter dem til HTTP-svar i standardformatet ProblemDetails (400, 404, 409). Uventede fejl giver 500 uden tekniske detaljer og logges på serveren.

Hvorfor: Controllerne har ingen try/catch, og alle fejl har samme format, så frontend kun skal håndtere ét. Application kender ikke HTTP, den beskriver kun, hvad der gik galt.

Alternativ: Result-mønsteret, hvor servicen returnerer et resultatobjekt i stedet for at kaste. Det er mere eksplicit, men kræver mere kode i hver controller.

Nullable felter i requests

Valg: Felterne i `Save...Request` er nullable (`string?`, `int?`, `bool?`, `DateOnly?`).

Hvorfor: Det opdagede jeg, da jeg byggede frontend. Et tomt datofelt blev afvist af ASP.NET med en teknisk engelsk fejl, før validatoren kørte. Med nullable felter når alle tomme felter frem til validatoren og giver beskeden "… er påkrævet" på dansk. Det gælder også `int?` og `bool?`, hvor 0 og `false` ellers ikke kunne skelnes fra "mangler".

Antal ansættelser på afsnit og personalegrupper

Valg: `DepartmentDto` og `StaffGroupDto` indeholder antal ansættelser, talt af databasen via en projektion. Datamodellen er uændret, og antallet gemmes ikke.

Hvorfor: Opgaven kræver filtrering på alle tabeller, men afsnit og personalegrupper har kun navn (og farve), som det ikke giver mening at filtrere på. Med antallet kan tabellerne filtreres på "Har ansættelser", og brugeren kan se, hvad der kan slettes, før de prøver.

Alternativer: - Ingen filtrering på afsnit og personalegrupper, kun søgning. Fravalgt, fordi opgaven kræver filtrering på alle tabeller. - Et gemt felt med antallet. Fravalgt, fordi det skulle holdes opdateret ved hver ændring af en ansættelse og kunne komme ud af takt.

Testdata

Valg: `DatabaseSeeder` indsætter opgavens eksempler og et par personer, der viser de interessante tilfælde, men kun når databasen er helt tom.

Hvorfor: Opgaven tillader indledende testdata i databasen. Data ligger i databasen og behandles som alle andre data, og seederen overskriver aldrig noget, brugeren selv har oprettet.

Navngivning: `Department` for afsnit

Valg: Afsnit hedder `Department` i koden og `departments` i API'et.

Hvorfor: Navnet blev valgt tidligt og bruges i entitet, DTO'er, service, controller, migration og frontend. Jeg opdagede navngivningen sent, og hvis jeg skulle omdøbe det ville det røre næsten hele løsningen og kræve en ny migration.

Begrænsning: Et afsnit er en del af en afdeling, og "department" betyder afdeling. Et mere præcist navn ville være `Unit eller Ward?`.

Kommentarer: `//` frem for `///`

Valg: Klasser og metoder har en kort `//`-kommentar, hvor navnet ikke siger det hele.

Hvorfor: Det er sådan, vi har gjort i mit team.

Alternativ: `///` (XML-dokumentationskommentarer), som er det normale i C#. De vises i IntelliSense, når man bruger metoden et andet sted, og kan bruges til at generere dokumentation. I et nyt projekt ville jeg følge teamets standard og ellers bruge `///`.

Controllere uden `[FromBody]`

Valg: Request-objekterne i `Create` og `Update` har ikke `[FromBody]`.

Hvorfor: Med `[ApiController]` henter ASP.NET automatisk objekter som `SavePeriodRequest` fra body'en og simple typer som `id` fra adressen. Templaten havde `[FromBody]` på `Create`, men ikke på `Update`, så jeg gjorde dem ens.

Alternativ: `[FromBody]` overalt, så det kan ses uden at kende reglen. Begge virker. Det afgørende er `[ApiController]`.

Oprydning efter templaten

- `WebAPI.http` er fjernet, fordi API'et testes i Swagger
- Templatens engelske README i roden er erstattet af en README for projektet, og den tomme `backend/README.md` er fjernet
- Den ubrugte pakke `Microsoft.AspNetCore.OpenApi` er fjernet. Swagger kører gennem `Swashbuckle.AspNetCore`
- Linjer i `.csproj`-filerne, der pegede på mapper, som ikke findes længere, er fjernet. Én af dem (`Compile Remove="Services\**"`) ville have fået compileren til at ignorere filer i en ny `Services`-mappe i Infrastructure uden fejl

Frontend

Ren klient-app (SPA)

Valg: React Router i SPA-tilstand (`ssr: false`).

Hvorfor: Det er et internt værktøj, så server-rendering for søgemaskiner og hurtig første visning har ingen værdi. Der er allerede en backend, så en ekstra Node-server ville kun tilføje kompleksitet.

Forbindelse til API'et: Vite-proxy

Valg: Under udvikling sender Vite alle kald til `/api` videre til backend.

Hvorfor: Frontend kender ikke backendens adresse, og backend skal ikke åbnes for andre adresser.

Alternativ: CORS i backend. Nødvendigt, hvis frontend og API skal køre på hver sin adresse i produktion. Så ville jeg konfigurere en snæver liste over tilladte adresser.

Data: `clientLoader` og `clientAction`

Valg: Hver side henter data i en `clientLoader` og modtager formularer i en `clientAction`. Dialogerne sender med `useFetcher` og et skjult felt `intent` (`create`, `update` eller `delete`).

Hvorfor: Det er sådan, React Router er bygget til at blive brugt, og efter hver handling henter den automatisk data igen, så tabellen altid er opdateret. React's dokumentation fraråder at hente data med `useEffect`, når man bruger et framework.

Egen tabelkomponent

Valg: Én generisk `DataTable<T>` med søgning, sortering og dropdown-filtre, som alle seks tabeller bruger. Hver side bestemmer kolonner og filtre.

Hvorfor: Alle seks tabeller skal kunne sorteres og filtreres, og alle undtagen Adgange skal kunne søges i. Med én fælles komponent opfører tabellerne sig ens, og en rettelse skal kun laves ét sted. Hver side vælger selv sine kolonner og filtre, og om der skal være søgning.

Alternativ: Et færdigt bibliotek som TanStack Table. Fravalgt, fordi opgaven kun kræver søgning, sortering og enkle filtre. Biblioteket har mange flere funktioner og ville gøre løsningen mere kompleks end den behøver at være.

Formularer og fejlvisning

Valg: - Én dialog til både opret og rediger, bygget på HTML's `<dialog>`, som giver tastaturfokus og Escape gratis - `FormDialog` samler det fælles (skjulte felter, fejlbesked, knapper), så hver formular kun indeholder sine felter - Browserens egen validering er slået fra (`noValidate`). Alle fejlbeskeder kommer fra backend, på dansk og vist ved det rigtige felt. `required` bliver stående, så stjernen vises, og skærmlæsere siger "påkrævet" - Knapper deaktiveres og viser "Gemmer…", mens et kald kører, så man ikke kan oprette det samme to gange - Sletning bekræftes i en dialog, som også viser backendens forklaring, hvis sletning afvises (409)

Filtrering

Valg: Dropdown-filtre på felter med få mulige værdier. Fritekst klares af søgefeltet.

Tabel	Filtre
Perioder	Status: Afsluttet / Igangværende / Kommende (beregnet ud fra dags dato)
Afsnit og Personalegrupper	Har ansættelser: Ja / Nej
Adgange	Admin: Ja/Nej, Personale: Ja/Nej (ingen søgning, som opgaven beskriver)
Ansættelser og Personale	Afsnit, Personalegruppe

Farvefelt

Valg: Browserens farvevælger og et HEX-felt side om side, som holder hinanden opdateret. Kun HEX-feltet sendes til backend.

Hvorfor: De fleste kender ikke HEX-koden for en farve, så et rent tekstfelt er dårligt for brugeren. Backend validerer stadig, fordi API'et ikke må stole på klienten.

Personale-siden

Valg: Den valgte periode står i adressen (`/staff?periodId=3`). Uden periode vises den igangværende.

Hvorfor: Når adressen ændres, henter React Router automatisk personalet for den nye periode. Valget overlever genindlæsning og kan deles som link.

Mindre valg

- **Dansk i brugerfladen, engelsk i koden og i adresserne.** Engelske adresser undgår æ, ø og å i URL'er og matcher API'et.
- **Ingen skrifttype fra Google Fonts.** Templaten hentede skrifttypen fra Googles servere, så brugernes browsere kontaktede Google. Jeg bruger systemets skrifttype i stedet.
- **Browserens eget datofelt**, så formatet følger brugerens egne indstillinger.
- **Templatens Docker-filer og frontend-README er fjernet**, fordi de beskrev en server-version af appen, som ikke findes i SPA-tilstand.
- **Dansk fejlside.** Templatens engelske "Oops!" er erstattet af en dansk side med "Prøv igen" og "Gå til forsiden". Tekniske detaljer vises kun under udvikling.
- **Én fælles `ColorDot`**, så afsnittets farve vises ens i Afsnit, Ansættelser og Personale.
- **Forældet `baseUrl` fjernet fra `tsconfig.json`** efter TypeScripts migreringsvejledning.

Fortolkninger af opgaven

Spørgsmål	Min fortolkning
Tæller slutdatoen med?	Ja. Opgavens eksempel med december og januar viser det
Må start og slut være samme dag?	Nej. "Start skal være før slut" og "slut skal være efter start" er strenge krav
Må to *personer* have overlappende ansættelser?	Ja. Overlap gælder kun den samme persons ansættelser
Hvad betyder "én eller begge roller"?	Rollerne er uafhængige: kun Admin, kun Personale eller begge. En bruger skal have mindst én
Kan Personale-rollen fjernes fra en bruger med ansættelser?	Nej. Så kunne der ligge ansættelser på en person, der ikke er personale
Er e-mail unik uanset store og små bogstaver?	Ja. E-mails gemmes med små bogstaver
Gyldig HEX-kode?	`#` og præcis seks tegn. Den korte form `#F00` er ikke tilladt, så alle farver gemmes ens
Må en periode med ansættelser slettes?	Ja. Ansættelser er knyttet til datoer, ikke til perioder, så intet går tabt
Hvordan vises en person med to ansættelser i samme periode?	Én række pr. ansættelse, fx ved skift af afsnit midt i perioden
Skal periodenavne være unikke?	Nej, opgaven kræver det ikke
Farve i tabellerne	Afsnittets farve vises som en prik ved navnet på Ansættelser og Personale. Ikke et krav, men det giver farven et formål

Kendte begrænsninger og næste skridt

1. **Opgradér til .NET 10** før .NET 8 udløber 10. november 2026
2. **Login** via organisationens eksisterende løsning (OpenID Connect, fx Microsoft Entra ID)
3. **SQL Server** i produktion og integrationstests mod SQL Server i pipelinen
4. **Forkert formaterede requests** (fx en dato som `"hej"`) afvises med ASP.NET's engelske standardfejl. De kunne oversættes til en dansk standardbesked
5. **Samtidig oprettelse af samme e-mail** fanges af det unikke indeks, men giver en 500. Den bør fanges og give 409
6. **Frontend-typerne** er skrevet i hånden og kunne genereres ud fra API'ets OpenAPI-beskrivelse, så de ikke kan komme ud af takt med backend
7. **Logikken i `DataTable`** (søgning, sortering, filtre) kunne flyttes til en separat fil og unit-testes, fx med Vitest
8. **Afhængigheder:** `npm audit` viser kendte sårbarheder i templatens pakker. De bør opdateres bevidst og testes bagefter
9. **Store datamængder:** søgning, sortering og paginering i API'et i stedet for i browseren
10. **Sletning af perioder:** hvis der senere lægges vagter ind i perioder, bør en periode med planlagte vagter ikke kunne slettes
11. **Navngivning:** omdøb `Department` til `Unit`, så koden bruger samme begreb som organisationen

Brug af AI

Jeg har brugt en sprogmodel (Claude) som sparringspartner og til kodeforslag under hele opgaven.

Jeg har stillet spørgsmål til de forslag jeg ikke forstod eller var enige i, og truffet beslutningerne undervejs.

Flere af valgene i beslutningsloggen er ændret eller tilføjet, fordi jeg udfordrede et forslag, fx validatorerne og farvevælgerne